

Домашнее задание № 2. Тема: «Объектно-ориентированные
возможности языка Си++»
Срок сдачи 10 неделя

Общие требования

Каждый класс должен поддерживать возможность чтения данных объекта из файла и получение данных с помощью генераторов псевдослучайных чисел (можно использовать конструкторы или методы для инициализации полей).

Примечание 1. В случае большого размера исходных данных рекомендуется получать их с помощью генератора псевдослучайных чисел.

Примечание 2. В случае, когда требуется добавить элемент к массиву, матрице и т.п., например, при перегрузке операции ++, и значение нового элемента не заданы, для получения значения использовать генератор псевдослучайных чисел.

Требования к отчету

Отчет должен содержать: условие задания, тексты файлов с исходным кодом программ. Результаты выполнения всех тестов: входные данные каждого теста и выходной результат теста.

Варианты заданий

Вариант 1

Определить класс CComplexVector для работы с вектором комплексных чисел.

Внутри класса вектор должен быть реализован с помощью указателя на тип std::complex. Длина вектора задается в конструкторе класса и изменяется, если происходит присваивание вектору другой длины.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), сложения, вычитания, скалярного умножения, << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину вектора. При сложении и вычитании длина результата - это минимум из длин исходных векторов. При скалярном умножении недостающие координаты считать нулевыми.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 2

Определить класс CIntN для работы с целыми беззнаковыми числами, состоящими из N десятичных цифр.

Внутри класса число должно быть реализовано с помощью указателя на тип char. Число N задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), сложения, вычитания, << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева) (++ и -- работают как умножение и деление на 10).

При сложении и вычитании количество значащих десятичных цифр результата (N) может отличаться от N аргументов.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс

(функция main).

Вариант 3

Определить класс CRats для работы с вектором несократимых дробей вида P_i/q_i , где P_i — целое, q_i — натуральное.

Внутри класса вектор должен быть реализован с помощью указателей (одного или двух). Длина вектора задается в конструкторе класса и изменяется, если происходит присваивание вектору другой длины.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), сложения, вычитания, << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину вектора.

При сложении и вычитании длина результата - это минимум из длин исходных векторов.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 4

Определить классы CPoints для работы с массивом точек на плоскости и CDists для работы с массивом расстояний между точками.

Внутри класса массивы должны быть реализованы с помощью указателей (одного или двух). Длина массива задается в конструкторе класса.

В классах должны быть определены необходимые конструкторы (в том числе конструкторы копирования и перемещения), деструкторы, операторы присваивания (копированием и перемещением), сложения (CPoints и CDists, возвращающий CPoints, а также CDists и CPoints, возвращающий CPoints), вычитания (CPoints из CPoints, возвращающий CDists), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину вектора. При сложении и вычитании длина результата - это минимум из длин исходных векторов.

При сложении CPoints и CDists или наоборот получает новые точки перемещением старых точек на заданное расстояние в направлении вектора с координатами (1, 1) и началом в (0, 0).

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 5

Определить класс CString для работы со строкой. Длина строки задается в конструкторе класса. Внутри класса строка должна быть реализована с помощью указателя (char *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), (добавление символа ! в конец строки и обрезание последнего символа), сложения (конкатенация), умножения (слева и справа) строки на беззнаковое целое число (оно равносильно сложению строки с собой нужное число раз). При сложении и умножении строк слагаемые и сомножители не должны изменяться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс

(функция main).

Вариант 6

Определить класс SAgg для работы с массивом целых чисел. Внутри класса массив должен быть реализован с помощью указателя. Длина массива задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину массива, сложения массива с целым числом (добавление числа в конец массива), При сложении массива с числом исходный массив не должен изменяться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 7

Определить класс CMatrix для работы с квадратной матрицей целых чисел. Внутри класса матрица должна быть реализована с помощью указателя (int **). Размер матрицы задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие размер матрицы, сложения, вычитания матриц, умножения матрицы (слева и справа) на число. При сложении и вычитании размер результата - это минимум из размеров исходных матриц, лишние строки и столбцы матрицы большего размера игнорируются.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 8

Определить класс CComplexMatrix для работы с матрицей комплексных чисел. Внутри класса матрица должна быть реализована с помощью указателя (complex **, класс complex тоже должен быть реализован самим, не библиотечный) или двух указателей double **. Размеры матрицы задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие размер матрицы, сложения и вычитания. При сложении и вычитании размер результата - это минимум из размеров исходных матриц, исходные матрицы при этом не должны меняться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 9

Определить класс CSet для работы с побитовым множеством целых чисел в диапазоне от 0 до N. Внутри класса множество должно быть реализовано с помощью указателя (unsigned int *). Число N задается в конструкторе класса. Принадлежность

числа множеству означает, что бит, соответствующей этому числу, равен 1, в случае нулевого бита число не принадлежит множеству.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), $\ll$ (вставка в поток вывода), инкремент и декремент $++$ и $--$ (справа и слева), увеличивающие и уменьшающие верхнюю границу диапазона, сложения (объединение множеств), вычитания (пересечения). При сложении верхняя граница диапазона результата равна максимуму из соответствующих границ слагаемых, при вычитании - минимуму. Исходные множества не должны при этих операциях меняться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция `main`).

Вариант 10

Определить класс `CPoly` для работы с полиномом от 2 переменных степени не выше N , N задается в конструкторе класса. Внутри класса полином должен быть реализован с помощью указателя (`double **`). Строки матрицы соответствуют степеням x , а столбцы степеням y . Значение элемента матрицы это коэффициент при умножении x и y в соответствующих степенях.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), $\ll$ (вставка в поток вывода, полином выводится по возрастанию степеней переменных), инкремент и декремент $++$ и $--$ (справа и слева) ($++$ и $--$ реализованы как дифференцирование по переменным x и y), операторы сложения, вычитания. При сложении и вычитании исходные полиномы не меняются.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция `main`) с распечаткой полинома по степеням переменных.

Вариант 11

Определить класс `CInt2` для работы с целыми беззнаковыми числами, состоящими из N двоичных цифр, N задается в конструкторе класса. Внутри класса число должно быть реализован с помощью указателя (`unsigned int *`).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), $\ll$ (вставка в поток вывода), инкремент и декремент $++$ и $--$ (справа и слева) ($++$ и $--$ реализованы как умножение и деление числа на 2), операторы сложения и вычитания (чисел в двоичной системе). Одна двоичная цифра должна занимать 1 бит. При сложении и вычитании исходные числа не меняются.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция `main`).

Вариант 12

Определить классы `CVects` для работы с массивом векторов на плоскости и `CAncls` для работы с массивом углов между векторами. Длина массива задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания

(копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину массива, операторы сложения (CVects и CAngls, возвращающий CVects, а также CAngls и CVects, возвращающий CVects), вычитания (CVects из CVects, возвращающий CAngls). При сложении CVects и CAngls или наоборот получаем новый CVects, путем поворота векторов на углы, положительное направление против часовой стрелки.

При сложении и вычитании длина результата - это минимум из длин исходных массивов, исходные массивы при этом не меняются.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 13

Определить класс CPoly для работы с многочленом от одной переменной с коэффициентами из поля вычетов Z_p , p - простое число, которое задается с помощью оператора #define. Степень многочлена задается в конструкторе класса. Внутри класса полином должен быть реализован с помощью указателя (int *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева) (результат - продифференцированный или проинтегрированный многочлен, при интегрировании произвольную постоянную считать равной 0), сложения и вычитания. При сложении и вычитании исходные полиномы не должны меняться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 14

Определить класс SAgt для работы с упорядоченным по возрастанию массивом целых чисел. Длина массива задается в конструкторе класса. Внутри класса массив должен быть реализован с помощью указателя (int *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину массива, сложения (массив, полученный слиянием слагаемых массивов). При сложении исходные массивы не должны меняться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 15

Определить класс SAgt для работы с массивом, содержащем элементы из поля вычетов Z_p , p - простое число, которое задается с помощью оператора #define целых чисел. Длина массива задается в конструкторе класса. Внутри класса массив должен быть реализован с помощью указателя (int *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину массива,

сложения массивов. Сложение выполняется поэлементно. В случае сложения массивов разной длины недостающие элементы считать равные 0.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 16

Определить класс SAgt для работы с упорядоченным по возрастанию массивом целых чисел. Длина массива задается в конструкторе класса. Внутри класса массив должен быть реализован с помощью указателя (int *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), дублирующий (удаляющий) последний элемент массива, сложения (массив, полученный пересечением слагаемых как мультимножеств).

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 17

Определить класс CMatrix для работы с квадратной матрицей, содержащей элементы из поля вычетов Z_p , p - простое число, которое задается с помощью оператора #define целых чисел. Внутри класса матрица должна быть реализована с помощью указателя (int **). Размер матрицы задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие размер матрицы, сложения матриц. Сложение выполняется поэлементно. При сложении размер результата - это минимум из размеров исходных матриц, лишние строки матрицы большего размера игнорируются.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 18

Определить класс CString для работы со строкой двухбайтовых символов Unicode. Длина строки задается в конструкторе класса. Внутри класса строка должна быть реализована с помощью указателя (unsigned short *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), (добавление символа ! в конец строки и обрезание последнего символа), сложения (конкатенация), умножения (слева и справа) строки на беззнаковое целое число (оно равносильно сложению строки с собой нужное число раз). При сложении и умножении строк слагаемые и сомножители не должны изменяться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 19

Определить класс `CAgg`, содержащий элементы из поля вычетов Z_p , p - простое число, которое задается с помощью оператора `#define`, целых чисел. Длина массива задается в конструкторе класса. Внутри класса массив должен быть реализован с помощью указателя (`int *`).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), `<<` (вставка в поток вывода), инкремент и декремент `++` и `--` (справа и слева), увеличивающие и уменьшающие длину массива, сложения и умножения, выполняющиеся над парами элементов массива. При сложении и умножении исходные массивы не должны меняться. При сложении или умножении размер результата - это минимум из размеров исходных массивов, лишние элементы массива большего размера игнорируются.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция `main`).

Вариант 20

Определить класс `CMatrix` для работы с прямоугольной матрицей вещественных чисел. Внутри класса матрица должна быть реализована с помощью указателя (`double **`). Размеры матрицы задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), `<<` (вставка в поток вывода), инкремент и декремент `++` и `--` (справа и слева), увеличивающие и уменьшающие размер матрицы, сложения, вычитания матриц, умножения матриц по правилам умножения матриц. В случае несоответствия размеров двух матриц выдается сообщение об ошибке.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция `main`).

Вариант 21

Определить класс `CPoly` для работы с многочленом от одной переменной с вещественными коэффициентами. Степень многочлена задается в конструкторе класса. Внутри класса полином должен быть реализован с помощью указателя (`double *`).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), `<<` (вставка в поток вывода), инкремент и декремент `++` и `--` (справа и слева) (результат - продифференцированный или проинтегрированный многочлен, при интегрировании произвольную постоянную считать равной 0), сложения и вычитания. При сложении и вычитании исходные полиномы не должны меняться.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция `main`).

Вариант 22

Определить класс `CVect`, содержащий вещественные элементы. Длина вектора задается в конструкторе класса. Внутри класса вектор должен быть

реализован с помощью указателя (double *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину вектора, сложения и скалярного произведения векторов. При сложении и произведении исходные вектора не должны меняться. В случае несоответствия размеров двух векторов выдается сообщение об ошибке.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 23

Определить класс CVect, содержащей вещественные элементы. Длина вектора задается в конструкторе класса. Внутри класса вектор должен быть реализован с помощью указателя (double *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину вектора, сложения и умножения вектора на число (справа и слева). При сложении и умножении исходные вектора не должны меняться. В случае несоответствия размеров двух векторов выдается сообщение об ошибке.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 24

Определить класс CVect, содержащей вещественные элементы. Длина вектора задается в конструкторе класса. Внутри класса вектор должен быть реализован с помощью указателя (double *).

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и декремент ++ и -- (справа и слева), увеличивающие и уменьшающие длину вектора, сложения и умножения вектора на число (справа и слева). При сложении и умножении исходные вектора не должны меняться. При сложении векторов длина нового вектора равна сумме длин векторов-слагаемых, вначале идут элементы первого вектора, затем второго.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).

Вариант 25

Определить класс CComplexMatrix для работы с матрицей комплексных чисел. Внутри класса матрица должна быть реализована с помощью указателя (complex **)/ Размер матрицы задается в конструкторе класса.

В классе должны быть определены необходимые конструкторы (в том числе конструктор копирования и перемещения), деструктор, операторы присваивания (копированием и перемещением), << (вставка в поток вывода), инкремент и

декремент ++ и -- (справа и слева), увеличивающие и уменьшающие размер матрицы, сложения и умножения. Умножение матриц выполняется по правилам умножения матриц. Исходные матрицы при сложении и умножении не должны меняться. В случае несоответствия размеров двух матриц выдается сообщение об ошибке.

Каждый класс должен быть описан в 2-х файлах (заголовочном и реализации). В отдельном файле должен быть написан тест на данный класс (функция main).